

Primera Entrega TP Final

Sistema de Gestión SAO

Curso de Inteligencia Artificial para Desarrolladores

Institución: UTN BA - Centro de e-Learning - 2026

Profesor: Ariel Garcia Traba - Emilio Esteban Rasic

Alumno: Lucas Jose Luccaroni

ÍNDICE:

1. Resumen ejecutivo y alcance técnico
2. Arquitectura de datos y concurrencia
 - Lógica de stock y bloqueo pesimista
 - Sincronización Realtime
3. Lógica de seguridad reforzada y roles
 - Jerarquía de roles
 - JornadaGuard y protección de terminales
 - Flujo de cierre y seguridad criptográfica
 - Seguridad a nivel de fila (RLS)
4. Diccionario de datos relacional
5. Módulo de reportes y analítica
6. Sistema de tokens visuales
 - Tipografía
 - Colores de marca
 - Colores funcionales
 - Colores por categoría
7. Especificaciones de interfaz (wireframes)
 - Layout general (Shell)
 - Módulo ABM de Productos
 - Terminal de Comandas
 - Historial y Reportes
8. Procedimientos operativos críticos
 - Envío de comanda (transacción atómica)
 - Flujo de cierre de jornada
9. Especificación técnica: Módulo Calculadora de Costos
10. Casos de uso
 - CDU-SAO-001: Tomar comanda
 - CDU-SAO-002: Seleccionar medio de pago
 - CDU-SAO-003: Ver historial reciente
 - CDU-SAO-004: Abrir jornada
 - CDU-SAO-005: Cerrar jornada
 - CDU-SAO-006: Gestionar productos
 - CDU-SAO-007: Validar PIN (SHA-256)
 - CDU-SAO-008: Cargar gastos
 - CDU-SAO-009: Auditoría de inventario
 - CDU-SAO-010: Ver historial jornadas
 - CDU-SAO-011: Monitorear caja (realtime)
 - CDU-SAO-012: Calculadora de costos

ANEXOS:

- A.** Diagrama entidad-relación (DER)
- B.** Diagrama de casos de uso (UML)
- C.** Flujo de venta — RPC procesar_comanda()
- D.** Flujo de cierre de jornada
- E.** UI Kit v2 — Sistema de tokens visuales
- F.** Diagrama de arquitectura
- G.** Wireframes de interfaz (v1.3)

DOCUMENTO MAESTRO DEFINITIVO V1.3: SISTEMA SAO BAR 2026

1. RESUMEN EJECUTIVO Y ALCANCE TÉCNICO

El sistema SAO Bar 2026 se define como una infraestructura web multi-terminal de alto rendimiento diseñada para la gestión operativa y administrativa de jornadas nocturnas. Su propósito central es garantizar la integridad transaccional y el control de inventario en un entorno de alta demanda, eliminando las discrepancias de los registros manuales.

El propósito por el cual elegí este proyecto para ser mi trabajo final del curso es para probar cómo desarrollar una app que puedo realizar con un stack tecnológico “no óptimo” utilizando las herramientas que conozco actualmente y llevar este proyecto a su máxima expresión en cuanto a estas cuestiones. Optimizar el diseño, el UX/UI, la robustez y la seguridad de las transacciones con la DB, todo esto con herramientas y lenguajes que actualmente desconozco pero que, con un flujo de trabajo cowork con IA agéntica, podré llevar a cabo. Una vez terminado el desarrollo, el siguiente nivel será aplicar una IA que audite sobre estos productos y el usuario administrador pueda consultarle sobre el sistema para tener un asistente dentro del mismo programa.

Concepto de Jornada Nocturna No-Calendario

Dada la naturaleza operativa del establecimiento, el sistema ignora la linealidad de la fecha calendario del servidor. La actividad que inicia un viernes por la noche y concluye la madrugada del sábado se agrupa bajo un identificador único de "Jornada". Esto permite consolidar métricas financieras y de stock sin la fragmentación que causaría un cambio de fecha a medianoche.

Entorno Operativo y Concurrencia

El sistema está dimensionado para la operación simultánea de 4 terminales activas bajo un modelo de datos centralizado:

- **3 Terminales de Comandas:** Dispositivos móviles o laptops destinados a la toma de pedidos en salón y barra por parte de los mozos.
- **1 Terminal de Administración:** Estación central (macOS) para el monitoreo de caja, auditoría física y cierre de jornada.

Stack Tecnológico (2026 Ready)

- **Framework:** Next.js 16 (App Router) para una gestión de rutas y estados optimizada.
- **Base de Datos:** Supabase (PostgreSQL) para persistencia relacional con integridad ACID.
- **Lógica de Servidor:** Server Actions para el procesamiento de reglas de negocio seguras.
- **Estilos:** TailwindCSS 4 para un diseño ligero, responsivo y de alta velocidad de renderizado.

2. ARQUITECTURA DE DATOS Y CONCURRENCIA

Para mitigar riesgos de sobreventa y colisiones de datos en un entorno de 4 terminales, la arquitectura traslada la lógica crítica al motor de la base de datos.

Lógica de Stock y Bloqueo Pesimista

El sistema implementa el Procedimiento Almacenado (RPC) `procesar_comanda()`. Este procedimiento ejecuta una transacción atómica que utiliza la cláusula **FOR UPDATE** (Bloqueo Pesimista). Al recibir un pedido, el sistema bloquea las filas de los productos involucrados, valida el stock disponible, descuenta las unidades y registra la venta. Si el stock es insuficiente, se ejecuta un Rollback inmediato, informando al operario sin corromper la base de datos.

Sincronización Realtime

Se utiliza **Supabase Realtime (WebSockets)** para la actualización instantánea de las interfaces de los mozos. Cuando el `stockActual` cambia en la base de datos tras una venta o reposición, todas las terminales reflejan el nuevo estado de los badges de stock sin necesidad de refrescar el navegador.

3. LÓGICA DE SEGURIDAD REFORZADA Y ROLES

Jerarquía de Roles

- **Rol Admin:** Acceso total, incluyendo Auditoría, Historial, ABM de Productos y Cierre de Caja.
- **Rol Empleado:** Acceso limitado exclusivamente a la toma de comandas y consulta de stock disponible.

JornadaGuard y Protección de Terminales

El sistema implementa el middleware **JornadaGuard**. Este componente bloquea la navegación y funcionalidad de las terminales si no existe una jornada marcada como "Abierta". Esto evita registros accidentales fuera del horario operativo.

Flujo de Cierre y Seguridad Criptográfica

La transición de una jornada de 'Cerrada' a 'Abierta' y de 'Abierta' a 'En Auditoría' es un punto crítico que bloquea preventivamente la creación de nuevas comandas en las 3 terminales de mozos. Este paso requiere la validación obligatoria del PIN de Administrador, el cual se procesa mediante un Hash **SHA-256** en el cliente, asegurando que la clave nunca viaje en texto plano.

Seguridad a Nivel de Fila (RLS)

Se implementa **Row Level Security (RLS)** en Supabase, garantizando que las políticas de acceso se ejecuten en el motor de la base de datos, blindando la información contra accesos no autorizados via API.

4. DICCIONARIO DE DATOS RELACIONAL (SIN TABLAS)

A continuación, se detalla la estructura relacional obligatoria para asegurar la integridad referencial.

- **Entidad: Usuarios**
 - id: UUID (Primary Key).
 - email: String (Único).
 - contraseña: Hash.
 - nombre: String.
 - rol: Enum ('Admin', 'Empleado').
 - pin: Hash SHA-256 (Exclusivo Admin).
- **Entidad: Productos**
 - id: UUID (Primary Key).

- nombre: String.
- categoría_id: Foreign Key (Categorías_Productos).
- precio: Numeric.
- stockIdeal: Integer (Referencia visual semanal).
- stockInicial: Integer (Actualizado solo en reposición o inicio).
- stockActual: Integer (Read-only, afectado por procesar_comanda()).
- unidad: String (u, lt, ml).
- activo: Boolean.
- **Entidad: Jornadas**
- jornada_id: UUID (Primary Key).
- estado: Enum ('abierta', 'en_auditoria', 'cerrada').
Constraint: no puede existir más de una jornada con estado 'abierta' o 'en_auditoria' simultáneamente. La apertura de una nueva jornada solo es válida cuando la jornada más reciente tiene estado 'cerrada'.
- fecha_inicio: Timestamp.
- fecha_fin: Timestamp.
- total_efectivo: Numeric.
- total_mp_lista: Numeric.
Representa el total teórico de ventas por Mercado Pago, calculado por el sistema en base a los precios de lista de los productos vendidos con medio_pago = 'Mercado Pago'. No contempla comisiones.
- total_mp_real: Numeric
Monto que efectivamente figura/acredita el posnet de Mercado Pago al momento del cierre de jornada. Es un dato ingresado manualmente por el Admin (no se calcula desde las comandas), ya que depende de variables externas al sistema (cuotas, tipo de cobro, condición de liquidez seleccionada por la clienta, etc.).
- comision_mp: Numeric.
Fórmula: $comision_mp = total_mp_lista - total_mp_real$. Se calcula automáticamente en el momento de la Consolidación Final del cierre y se persiste en la base de datos.
- total_general: Numeric.(total_efectivo + total_mp_real)
- gastos_totales: Numeric.
- ganancia_neta: Numeric.($ganancia_neta = total_efectivo + total_mp_real - gastos_totales$.)
- **Entidad: Comandas**
- comanda_id: UUID (Primary Key).
- jornada_id: Foreign Key (Jornadas).
- usuario_id: Foreign Key (Usuarios).
- numero_ticket: SERIAL (Incremental para control de barra/cocina)
- nro_beeper: Integer (Opcional).
- fecha: Timestamp.
- total: Numeric.
- medio_pago: Enum ('Efectivo', 'Mercado Pago').
- **Entidad: Comanda_Items (N:N Break)**
- id: UUID (Primary Key).
- comanda_id: Foreign Key (Comandas).
- producto_id: Foreign Key (Productos).
- cantidad: Integer.
- precio_unitario_historico: Numeric (Snapshot del precio al momento de la venta).
- **Entidad: Gastos**
- id: UUID (Primary Key).
- jornada_id: Foreign Key (Jornadas).
- categoría_id: Foreign Key (Categorías_Gastos).

- descripción: String.
- monto: Numeric.
- **Entidad: Auditoria_Inventario**
- id: UUID (Primary Key).
- jornada_id: Foreign Key (Jornadas).
- producto_id: Foreign Key (Productos).
- conteo_fisico: Integer (Ingresado por Admin).

5. MÓDULO DE REPORTES Y ANALÍTICA

Generación Dinámica y Auditoría Anti-Robo

Los reportes no son estáticos; se generan mediante **JOINS relacionales** en tiempo real. El motor de auditoría aplica la siguiente fórmula de conciliación:

- **Stock Teórico = Stock Inicial - Unidades Vendidas (Agregado de las 3 terminales).**

Visualización de Desvíos y Ordenamiento

- **Token Error:** Si el Conteo Físico difiere del Stock Teórico, la interfaz resalta el registro con el color funcional Error (#E2484A).
- **Jerarquía de Datos:** Los reportes de stock se ordenan descendentemente por volumen de ventas, priorizando la visualización de los productos de mayor rotación.

Conciliación de Mercado Pago

Además de la fórmula de conciliación de stock ya existente (Stock Teórico = Stock Inicial - Unidades Vendidas), se incorpora una conciliación financiera específica para Mercado Pago, visible en el detalle de cada jornada cerrada (módulo Historial y Reportes):

- Total MP Lista: total_mp_lista (teórico, según precios de venta registrados en comandas).
- Total MP Real: total_mp_real (monto real informado por el posnet, ingresado por el Admin).
- Comisión MP: comision_mp = total_mp_lista - total_mp_real.

Token Advertencia: si comision_mp supera un porcentaje configurable del total_mp_lista (a definir junto con la clienta, ej. 8-10%), el valor de Comisión MP se resalta con el color funcional Advertencia (#BA7517), para alertar de una comisión fuera de lo habitual (posible error de tipo o condición de cobro distinta a la esperada).

6. SISTEMA DE TOKENS VISUALES "T" (SIN TABLAS)

Tipografía (Google Fonts)

- **Creepster:** Estilo Display para Logotipo "SAO Bar" y elementos decorativos de marca.
- **Livvic:** Estilo UI para toda la interfaz (botones, tablas, inputs y lectura).

Colores de Marca

- Negro: #080A0D (Fondo primario).
- Naranja: #F26A1B (Acciones primarias y paneles).

- Naranja Alternativo: #F25922 (Interacciones y estados activos).
- Cyan: #30CFF2 (Énfasis de navegación y logo).
- Blanco: #F2F2F2 (Texto principal y contrastes).

Colores Funcionales

- Efectivo: #1D9E75 (Confirmación y éxito).
- Mercado Pago: #378ADD (Pagos digitales).
- Error/Peligro: #E2484A (Desvíos de stock, alertas y errores).
- Advertencia: #BA7517 (Stock bajo).
- Gris UI: #9D9D9D (Textos secundarios y deshabilitados).

Colores por Categoría

- Cerveza: Fondo #2A1F0A / Texto #E6A830.
- Trago: Fondo #1A1F3A / Texto #7B9CF6.
- Sin Alcohol: Fondo #0E2A1A / Texto #4DB87A.
- Comida: Fondo #2A1A1A / Texto #D97C7C.

7. ESPECIFICACIONES DE INTERFAZ (WIREFRAMES DESCRIPTIVOS)

Layout General (Shell)

- **Topnav (56px):** Navegación global con indicadores de rol.
- **Barra de Título (56px):** Contexto de pantalla y acciones rápidas.

Módulo ABM de Productos (Patrón Master-Detail)

- **Panel Izquierdo (1140px):** Grilla de productos con 8 columnas. Los productos marcados como activo: false deben renderizarse con un **40% de opacidad**.
- **Panel Derecho (300px):** Formulario de edición/alta. El campo stockActual debe ser siempre de solo lectura con fondo #141414.

Terminal de Comandas

- **Sidebar Izquierdo (80px):** Selector de categorías vertical.
- **Grilla de Productos:** Cards con badges de stock dinámicos.
- **Carrito Flotante:** Resumen de pedido con botones de pago rápido (Efectivo/MP).
- **Historial de comanda:** Pequeño historial visible muestra las últimas N comandas de la jornada activa sin filtro por usuario.

Cierre de jornada

- **Sidebar Izquierdo (80px):** Selector de categorías vertical
- **Wireframes descriptivos del cierre adjuntos:** Además de seguir el diseño ya establecido en los wireframes generados, en la sección de “Carga de gastos” se agrega la sección “Mercado Pago - Cierre de Posnet: un input numérico único para que el Admin ingrese el monto que figura en el posnet de Mercado Pago (total_mp_real).
 - Debajo del input, en modo solo lectura, se muestra el Total MP Lista (total_mp_lista) calculado por el sistema, para que el Admin pueda comparar ambos valores antes de confirmar.
 - El campo total_mp_real es obligatorio para poder avanzar al siguiente paso (Auditoría de Inventario) — no se puede dejar vacío ni en 0 si hubo comandas con medio_pago = 'Mercado Pago' en la jornada.

- - Estilo: mismo patrón visual que los inputs de Gastos (fondo Negro Profundo #1A1A1A, borde Gris UI #9D9D9D, foco Naranja Alternativo #F25922).

Historial y Reportes

- **Master-Detail de Auditoría:** Columna izquierda de jornadas con un ancho de **400px**. El detalle derecho muestra la tabla de stock con el ordenamiento por ventas y resaltado de errores.

8. PROCEDIMIENTOS OPERATIVOS CRÍTICOS

Envío de Comanda (Transacción Atómica)

Al invocar procesar_comanda():

1. Validación de estado de jornada (Debe ser 'Abierta').
2. Bloqueo de filas en productos vía FOR UPDATE.
3. Validación de disponibilidad (stockActual >= cantidad).
4. Inserción en comandas.
5. Inserción en comanda_items (Capturando precio actual).
6. Deducción del stockActual.

Flujo de Cierre de Jornada

Este proceso debe ser ejecutado en orden estrictamente atómico:

1. **Validación de PIN:** El Administrador ingresa su PIN (Hash SHA-256).
2. **Transición de Estado:** Cambio de la jornada a 'En Auditoría', bloqueando todas las terminales de mozos mediante JornadaGuard.
El estado en_auditoria no tiene timeout. Una jornada puede permanecer en este estado indefinidamente sin comprometer la integridad de los datos.
A nivel de base de datos, debe existir una constraint que impida la inserción de una nueva jornada si ya existe algún registro con estado abierta o en_auditoria. La apertura de una nueva jornada solo es válida cuando la jornada más reciente tiene estado cerrada.
3. **Carga de Gastos + Stock real MP:** Registro de egresos en la entidad Gastos, Y carga del monto total_mp_real informado por el posnet de Mercado Pago (ambos en la misma pantalla/paso). El sistema muestra en simultáneo el total_mp_lista calculado, para contraste visual.
4. **Auditoría Física:** Carga del conteo manual en Auditoria_Inventario.
5. **Consolidación Final:** El sistema calcula totales:
 - $\text{comision_mp} = \text{total_mp_lista} - \text{total_mp_real}$ (se persiste en Jornadas).
 - $\text{total_general} = \text{total_efectivo} + \text{total_mp_real}$.
 - $\text{ganancia_neta} = \text{total_efectivo} + \text{total_mp_real} - \text{gastos_totales}$.
 La jornada pasa a estado 'Cerrada'.

9. Especificación técnica: Módulo Calculadora de Costos (Stateless)

a. Resumen Funcional y Objetivos del Módulo

La Calculadora de Costos se define como una herramienta de utilidad volátil y de alta fidelidad dentro del ecosistema SAO Bar 2026. Su propósito es permitir a la administración

(dueña/administradores) realizar simulaciones rápidas de costos de insumos sin impactar los registros oficiales de inventario o finanzas. **Naturaleza Técnica:** El módulo es estrictamente **Stateless** respecto a la persistencia en servidor. La totalidad del estado dinámico reside en la memoria volátil del cliente (React State). Al refrescar la aplicación o navegar fuera del módulo, los datos se eliminan. No existe integración con las tablas de PostgreSQL en Supabase para este componente.

Aplicación de Tokens Visuales (\$T\$) y Consistencia Estética

La interfaz debe adherirse rigurosamente al sistema de diseño del proyecto para garantizar una transición imperceptible desde el Shell principal.

- **Tipografía Obligatoria:**
- **Títulos y Branding:** Creepster (vía Google Fonts).
- **Interfaz, Inputs y Datos:** Livvic (vía Google Fonts).
- **Paleta de Colores (Tokens \$T\$):**
- **Fondo de Contenedor Principal:** Naranja (#F26A1B).
- **Fondo de Inputs y Áreas de Trabajo:** Negro Profundo (#1A1A1A).
- **Bordes de UI (Trazo):** Gris UI (#9D9D9D) - 1px solid.
- **Estados Activos / Foco de Input:** Naranja Alternativo (#F25922).
- **Resaltado de Totales:** Cyan (#30CFF2).
- **Acciones de Guardado/Confirmación (Simuladas):** Blanco (#F2F2F2) con texto Negro (#080A0D).
- **Acciones de Eliminación / Errores:** Rojo Error (#E2484A).

b. Arquitectura del Componente y Gestión de Estado

El módulo se implementará como un **Client Component** de Next.js 16 dentro de la estructura del App Router (app/(admin)/calculadora/page.tsx).

c. Definición de Tipos (TypeScript)

```
interface CostRow
{
  id: string // generado via crypto.randomUUID()
  insumo: string,
  cantidad: number,
  precioUnitario: number;
}
```

d. Lógica de Estado Local

Se utilizará el hook useState para gestionar el array de filas transitorias.

- **Inicialización:** El estado inicial debe contener una fila vacía para incentivar la interacción inmediata.
- **Adición de Filas:** Función que inyecta un nuevo objeto CostRow al array.
- **Eliminación:** Filtrado del array de estado mediante el ID único de la fila.
- **Controlled Inputs:** Implementación de manejadores de eventos que actualicen propiedades específicas del objeto dentro del array de estado mediante el índice o ID.

e. Algoritmo de Cálculo y Computación Local

Para optimizar el rendimiento y evitar re-renders innecesarios, se debe utilizar el hook useMemo para computar los totales.

- **Subtotal por Fila:** cantidad * precioUnitario.
- **Total General:** Implementar un .reduce() sobre el array de CostRow.

- **Formateo:** Los resultados deben presentarse en formato moneda (ARS) utilizando Intl.NumberFormat('es-AR', { style: 'currency', currency: 'ARS' }).
- **Visualización de Resultados:** El total general debe renderizarse en el pie del contenedor utilizando Livvic Bold, color Cyan (#30CFF2).

f. Diseño de Interfaz y Layout

El layout debe respetar las dimensiones del Shell global para evitar saltos de contenido (Layout Shift).

- **Integración de Shell:**
 - Topnav: 56px.
 - Title Bar (Barra de Título): 56px.
 - Altura del contenedor de la calculadora: min-height: calc(100vh - 112px).
- **Disposición de Tabla:**
 - Encabezados: Nombre del Insumo, Cantidad, Precio Unitario, Subtotal, Acciones.
 - Estilo de Filas: Fondo Negro Profundo (#1A1A1A) con bordes Gris UI (#9D9D9D).
 - Inputs: Sin bordes nativos, utilizando el token Naranja Alternativo (#F25922) en el estado :focus.

g. Manejo de Errores y Validaciones en Cliente

El frontend debe actuar como el único validador de la integridad de los datos simulados:

- **Validación Numérica:** Bloqueo absoluto de valores negativos mediante Math.max(0, value) en los controladores de eventos.
- **Campos Vacíos:** Los cálculos de subtotal deben retornar 0 en lugar de NaN si los campos numéricos están vacíos.
- **Feedback Visual:** Aplicación del token Rojo Error (#E2484A) en los bordes de los inputs si se detectan valores inconsistentes.

h. Protocolo de Persistencia Inexistente e Impresión

Dado que no existe una tabla calculadora_costos en Supabase, el sistema debe gestionar las expectativas del usuario.

- **Advertencia de Volatilidad:** Incluir un banner persistente en la parte superior del módulo con el texto: *"Aviso: Esta calculadora es una herramienta de simulación. Los datos no se guardan en la base de datos y se perderán al cerrar la pestaña."*
- **Protocolo de "Guardado":** El botón principal de acción no realizará un fetch. En su lugar, ejecutará window.print().
- **Estilos de Impresión (@media print):**
 - Ocultar: Topnav, Barra de Título, Sidebar (si existiera) y botones de "Acciones" (eliminar fila).
 - Mostrar: Tabla de costos sobre fondo blanco con texto negro para ahorro de tinta, manteniendo el logo de SAO Bar en la parte superior.

i. Control de Acceso (Route Guard)

Aunque los datos sean locales, el acceso al módulo debe estar restringido mediante el ProtectedRoute del sistema, validando que el usuario posea el rol de **admin**. Si un usuario con rol de empleado intenta acceder via URL, debe ser redirigido al dashboard de Comandas.

10. Casos de uso

Caso de Uso: Tomar comanda	
Caso de uso ID	CDU-SAO-001
Nombre	Tomar comanda
Actor/es	Empleado
Descripción	El empleado registra el pedido de un cliente seleccionando productos del menú, los agrega al carrito y confirma la comanda con el medio de pago correspondiente. El sistema valida stock y registra la venta de forma atómica.
Precondiciones	Existe una jornada en estado 'abierta'. El empleado tiene sesión iniciada con rol Empleado. Los productos seleccionados tienen stockActual >= cantidad solicitada.
Postcondiciones	Se registra una nueva Comanda en la base de datos. Se registran los Comanda_Items correspondientes con precio_unitario_historico. El stockActual de cada producto es decrementado. Se emite un número de ticket SERIAL único para el turno. Todas las terminales reflejan el nuevo stock vía Realtime.
Prioridad	Alta
Frecuencia de uso	Muy alta — operación central durante toda la jornada.
Flujo normal	1. El empleado accede a la terminal de comandas (jornada abierta). 2. Selecciona la categoría de producto desde el sidebar. 3. Selecciona productos y cantidades desde la grilla. 4. Revisa el carrito flotante con el resumen del pedido. 5. Ingresa nro_beeper si corresponde. 6. Selecciona medio de pago (Efectivo o Mercado Pago). 7. Pulsa 'Pagar'. El sistema invoca procesar_comanda() via Server Action. 8. El sistema confirma con número de ticket y vacía el carrito.
Flujos alternativos	Stock insuficiente: el RPC ejecuta ROLLBACK, se muestra toast de error, el carrito permanece intacto. El empleado elimina un ítem del carrito antes de confirmar.
Excepciones	JornadaGuard bloquea la UI si la jornada no está en estado 'abierta'. Error de conexión: el Server Action retorna error, no se modifica la base de datos.
Includes	CDU-SAO-002 — Seleccionar medio de pago
Extends	-/-
Requerimientos no funcionales	La transacción debe completarse en menos de 2 segundos bajo concurrencia de 4 terminales. El bloqueo pesimista (FOR UPDATE) garantiza integridad ante ventas simultáneas.
Notas	El historial de las últimas N comandas de la jornada es visible en la terminal sin filtro por usuario.

Caso de Uso: Seleccionar medio de pago	
Caso de uso ID	CDU-SAO-002
Nombre	Seleccionar medio de pago
Actor/es	Empleado
Descripción	Sub-proceso incluido en Tomar comanda. El empleado elige entre Efectivo o Mercado Pago antes de confirmar la comanda. El valor se envía como parámetro al RPC procesar_comanda().
Precondiciones	Existe al menos un producto en el carrito. La jornada está en estado 'abierta'.
Postcondiciones	El campo medio_pago de la Comanda queda registrado con el valor seleccionado. El total se acumula en total_efectivo o total_mp_lista de la Jornada al cierre.
Prioridad	Alta
Frecuencia de uso	Ocurre en cada comanda.
Flujo normal	1. El empleado visualiza los botones 'Efectivo' y 'Mercado Pago' en el carrito flotante. 2. Pulsa el botón correspondiente. 3. El valor queda seleccionado y se incluye en el payload al confirmar.
Flujos alternativos	-/-
Excepciones	Si no se selecciona medio de pago, el botón 'Pagar' permanece deshabilitado.
Includes	-/-
Extends	-/-
Requerimientos no funcionales	Los botones de pago deben ser visualmente diferenciados por color token: Efectivo #1D9E75, Mercado Pago #378ADD.
Notas	-/-

Caso de Uso: Ver historial reciente	
Caso de uso ID	CDU-SAO-003
Nombre	Ver historial reciente
Actor/es	Empleado
Descripción	El empleado consulta las últimas N comandas registradas en la jornada activa. La vista es de solo lectura y no filtra por usuario, permitiendo coordinación entre mozos.
Precondiciones	Existe una jornada en estado 'abierta'. El empleado tiene sesión iniciada.
Postcondiciones	Ninguna — operación de solo lectura.
Prioridad	Media
Frecuencia de uso	A demanda durante la jornada.
Flujo normal	1. El empleado accede al historial visible en la terminal de comandas. 2. El sistema muestra las últimas N comandas de la jornada con número de ticket, ítems, total y medio de pago.
Flujos alternativos	No hay comandas en la jornada: se muestra estado vacío.
Excepciones	JornadaGuard bloquea el acceso si no hay jornada abierta.
Includes	-//-
Extends	-//-
Requerimientos no funcionales	La lista se actualiza en tiempo real vía Supabase Realtime sin recargar la página.
Notas	-//-

Caso de Uso: Abrir jornada	
Caso de uso ID	CDU-SAO-004
Nombre	Abrir jornada
Actor/es	Admin
Descripción	El administrador inicia una nueva jornada nocturna. El sistema crea el registro en la tabla Jornadas con estado 'abierta', habilitando las terminales de mozos. Requiere validación de PIN.
Precondiciones	No existe ninguna jornada en estado 'abierta' o 'en_auditoria' (constraint UNIQUE partial). El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Se crea un registro en Jornadas con estado 'abierta' y fecha_inicio. Las terminales de mozos quedan habilitadas (JornadaGuard permite navegación).
Prioridad	Alta
Frecuencia de uso	Una vez por jornada nocturna.
Flujo normal	1. El administrador accede al panel de control. 2. Pulsa 'Abrir jornada'. 3. El sistema solicita PIN de administrador. 4. El administrador ingresa el PIN (hash SHA-256 generado en cliente). 5. El Server Action valida el hash contra la base de datos. 6. El sistema crea la jornada y habilita las terminales.
Flujos alternativos	-/-
Excepciones	PIN inválido: se muestra error, se permite reintento. Ya existe jornada abierta: el sistema informa y bloquea la acción.
Includes	CDU-SAO-007 — Validar PIN (SHA-256)
Extends	-/-
Requerimientos no funcionales	El PIN nunca viaja en texto plano — se hashea con SHA-256 en el cliente antes del envío.
Notas	-/-

Caso de Uso: Cerrar jornada	
Caso de uso ID	CDU-SAO-005
Nombre	Cerrar jornada
Actor/es	Admin
Descripción	El administrador ejecuta el cierre completo de la jornada nocturna en pasos secuenciales y atómicos: validación de PIN, transición a 'en_auditoria', carga de gastos y monto real de Mercado Pago (posnet), auditoría física de inventario y consolidación final.
Precondiciones	Existe una jornada en estado 'abierta'. El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Postcondiciones: La jornada queda en estado 'cerrada'. Se registran total_efectivo, total_mp_lista, total_mp_real, comision_mp, gastos_totales y ganancia_neta. Se registra el conteo físico en Auditoria_Inventario. Las terminales de mozos quedan bloqueadas (JornadaGuard activo).
Prioridad	Alta
Frecuencia de uso	Una vez al finalizar cada jornada nocturna.
Flujo normal	1. El administrador inicia el proceso de cierre. 2. Ingresa y valida el PIN (SHA-256). 3. El sistema transiciona la jornada a 'en_auditoria' — terminales bloqueadas. 4. . El administrador carga los gastos de la jornada y el monto real informado por el posnet de Mercado Pago (total_mp_real). 5. El administrador ingresa el conteo físico de inventario. 6. El sistema consolida totales (incluida la comisión MP) y pasa la jornada a 'cerrada'.
Flujos alternativos	El administrador cancela en algún paso intermedio: la jornada queda en 'en_auditoria' hasta completar el proceso.
Excepciones	PIN inválido: error con posibilidad de reintento. Error en consolidación: se registra sin modificar el estado de 'en_auditoria'.
Includes	CDU-SAO-007 — Validar PIN (SHA-256) CDU-SAO-008 — Cargar gastos y monto real MP CDU-SAO-009 — Auditoría inventario
Extends	-/-
Requerimientos no funcionales	El proceso es estrictamente secuencial y atómico. El cambio de estado a 'en_auditoria' bloquea preventivamente todas las terminales de mozos. El campo total_mp_real es obligatorio si existieron comandas con medio_pago = 'Mercado Pago' en la jornada.
Notas	-/-

Caso de Uso: Gestionar productos	
Caso de uso ID	CDU-SAO-006
Nombre	Gestionar productos
Actor/es	Admin
Descripción	El administrador realiza altas, bajas lógicas y modificaciones sobre el catálogo de productos mediante el módulo ABM con patrón Master-Detail.
Precondiciones	El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Los cambios quedan persistidos en la tabla Productos. Los productos con activo: false se renderizan con 40% de opacidad en todas las terminales. El stockActual nunca es modificable desde este módulo (campo read-only).
Prioridad	Media
Frecuencia de uso	A demanda — gestión administrativa.
Flujo normal	1. El administrador accede al módulo ABM de Productos. 2. Visualiza la grilla de productos con 8 columnas. 3. Selecciona un producto para editar (panel derecho se activa) o crea uno nuevo. 4. Completa/modifica los campos del formulario. 5. Guarda los cambios.
Flujos alternativos	Baja lógica: el administrador desactiva el producto (activo: false) sin eliminarlo.
Excepciones	Intento de modificar stockActual: el campo es read-only y no acepta entrada.
Includes	-/-
Extends	-/-
Requerimientos no funcionales	El campo stockActual debe tener fondo #141414 y ser siempre de solo lectura. Los productos inactivos se renderizan con 40% de opacidad.
Notas	El stockActual solo es afectado por el RPC procesar_comanda().

Caso de Uso: Validar PIN (SHA-256)	
Caso de uso ID	CDU-SAO-007
Nombre	Validar PIN (SHA-256)
Actor/es	Admin
Descripción	Sub-proceso de seguridad incluido en Abrir jornada y Cerrar jornada. El PIN del administrador es hasheado con SHA-256 en el cliente y comparado contra el hash almacenado en la base de datos. El PIN nunca viaja en texto plano.
Precondiciones	El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Si el hash coincide: el flujo padre continúa. Si no coincide: se notifica el error y se permite reintento.
Prioridad	Alta
Frecuencia de uso	Cada vez que se abre o cierra una jornada.
Flujo normal	1. El sistema presenta el campo de ingreso de PIN. 2. El administrador ingresa su PIN. 3. El cliente genera el hash SHA-256 del PIN ingresado. 4. El Server Action compara el hash con el valor almacenado en Usuarios.pin. 5. Si coincide, retorna validación exitosa al flujo padre.
Flujos alternativos	--
Excepciones	Hash no coincide: error visible, reintento habilitado. Sin conexión: el Server Action no puede validar, se muestra error de red.
Includes	--
Extends	--
Requerimientos no funcionales	El hash SHA-256 se genera exclusivamente en el cliente mediante Web Crypto API. El PIN en texto plano nunca sale del dispositivo.
Notas	--

Caso de Uso: Cargar gastos y monto real MP	
Caso de uso ID	CDU-SAO-008
Nombre	Cargar gastos
Actor/es	Admin
Descripción	Sub-proceso del cierre de jornada. El administrador registra los egresos de la jornada categorizados. Y el monto real informado por el posnet de Mercado Pago al cierre. Solo disponible cuando la jornada está en estado 'en_auditoria'.
Precondiciones	La jornada está en estado 'en_auditoria'. El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Los gastos quedan registrados en la tabla Gastos con su categoría correspondiente. El total de gastos se acumula para la consolidación final de la jornada. El campo total_mp_real de la Jornada queda registrado con el valor ingresado por el Admin. El sistema calcula y persiste $comision_mp = total_mp_lista - total_mp_real$.
Prioridad	Alta
Frecuencia de uso	Una vez por proceso de cierre de jornada.
Flujo normal	1. El sistema presenta el formulario de carga de gastos junto con el campo de monto real de Mercado Pago. 2. El administrador ingresa descripción, monto y categoría de cada gasto. Puede agregar múltiples gastos. 3. El administrador ingresa el monto total_mp_real visualizado en el posnet de Mercado Pago. 4. El sistema muestra en paralelo el total_mp_lista (calculado por el sistema) para que el Admin compare ambos valores. 5. El administrador confirma la carga. 6. El sistema registra los gastos en la tabla Gastos y persiste total_mp_real y comision_mp en la Jornada.
Flujos alternativos	No hay gastos en la jornada: el administrador avanza sin registrar ninguno (pero igual debe ingresar total_mp_real si hubo ventas por MP).
Excepciones	Excepciones: Error al persistir: se muestra notificación y se permite reintentar. Si hubo comandas con medio_pago = 'Mercado Pago' en la jornada y total_mp_real queda vacío o en 0, el sistema bloquea el avance al siguiente paso.
Includes	-/-
Extends	-/-
Requerimientos no funcionales	Requerimientos no funcionales: Solo accesible cuando jornada.estado = 'en_auditoria'. El campo total_mp_real es de ingreso manual (no editable por el sistema), dado que depende de condiciones externas (cuotas, tipo de cobro, liquidez seleccionada, etc.) que el sistema no puede calcular.
Notas	comision_mp puede ser mayor a 0 (Mercado Pago retuvo comisión) o, en casos o igual a 0. No debería ser negativo; si el Admin ingresa un total_mp_real mayor al total_mp_lista, el sistema debe advertir (no necesariamente bloquear) ya que sería un caso atípico.

Caso de Uso: Auditoría inventario	
Caso de uso ID	CDU-SAO-009
Nombre	Auditoría inventario
Actor/es	Admin
Descripción	Sub-proceso del cierre de jornada. El administrador ingresa el conteo físico real de cada producto. El sistema calcula el desvío comparando contra el Stock Teórico (stockInicial - unidades vendidas) y resalta diferencias.
Precondiciones	La jornada está en estado 'en_auditoria'. El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Se registra el conteo_fisico por producto en la tabla Auditoria_Inventario. El sistema calcula: Stock Teórico = stockInicial - unidades vendidas. Los productos con desvío se resaltan con color Error (#E2484A).
Prioridad	Alta
Frecuencia de uso	Una vez por proceso de cierre de jornada.
Flujo normal	1. El sistema presenta la lista de productos ordenada por volumen de ventas descendente. 2. El administrador ingresa el conteo físico de cada producto. 3. El sistema calcula en tiempo real el desvío respecto al Stock Teórico. 4. El administrador confirma la auditoría. 5. El sistema persiste los datos en Auditoria_Inventario.
Flujos alternativos	-/-
Excepciones	Error al persistir: se muestra notificación y se permite reintentar.
Includes	-/-
Extends	-/-
Requerimientos no funcionales	Desvíos resaltados con token color Error (#E2484A). Productos ordenados descendientemente por volumen de ventas para priorizar los de mayor rotación.
Notas	La fórmula de conciliación es: Stock Teórico = stockInicial - Σ (unidades vendidas en comandas de la jornada).

Caso de Uso: Ver historial jornadas	
Caso de uso ID	CDU-SAO-010
Nombre	Ver historial jornadas
Actor/es	Admin
Descripción	El administrador consulta el historial de jornadas cerradas con sus métricas financieras y el detalle de auditoría de stock por jornada seleccionada.
Precondiciones	El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Ninguna — operación de solo lectura.
Prioridad	Media
Frecuencia de uso	A demanda — revisión histórica.
Flujo normal	1. El administrador accede al módulo de Historial y Reportes. 2. Visualiza la columna izquierda (400px) con lista de jornadas cerradas. 3. Selecciona una jornada. 4. El panel derecho muestra la tabla de stock con desvíos y métricas financieras.
Flujos alternativos	No hay jornadas cerradas: se muestra estado vacío.
Excepciones	-//-
Includes	-//-
Extends	-//-
Requerimientos no funcionales	Los reportes se generan mediante JOINS relacionales en tiempo real, no son estáticos. Los desvíos se resaltan con color Error (#E2484A).
Notas	-//-

Caso de Uso: Monitorear caja (realtime)	
Caso de uso ID	CDU-SAO-011
Nombre	Monitorear caja (realtime)
Actor/es	Admin
Descripción	El administrador visualiza en tiempo real los totales acumulados de la jornada activa: efectivo, Mercado Pago, cantidad de comandas y estado general de caja.
Precondiciones	Existe una jornada en estado 'abierta'. El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Ninguna — operación de solo lectura.
Prioridad	Alta
Frecuencia de uso	Continua durante la jornada.
Flujo normal	1. El administrador accede al panel de monitoreo de caja. 2. El sistema muestra totales actualizados en tiempo real vía Supabase Realtime. 3. Los valores se actualizan automáticamente al registrarse nuevas comandas.
Flujos alternativos	-//-
Excepciones	Pérdida de conexión WebSocket: los datos dejan de actualizarse en tiempo real. Se muestra indicador de desconexión.
Includes	-//-
Extends	-//-
Requerimientos no funcionales	Actualización en tiempo real vía WebSockets (Supabase Realtime). Sin necesidad de refrescar la página.
Notas	-//-

Caso de Uso: Calculadora de costos	
Caso de uso ID	CDU-SAO-012
Nombre	Calculadora de costos
Actor/es	Admin
Descripción	Herramienta stateless de simulación de costos de insumos. Permite al administrador calcular costos de forma volátil sin impactar la base de datos. Los datos se pierden al cerrar la pestaña.
Precondiciones	El administrador tiene sesión iniciada con rol Admin.
Postcondiciones	Ninguna — el módulo es stateless. No persiste datos en Supabase.
Prioridad	Baja
Frecuencia de uso	A demanda.
Flujo normal	1. El administrador accede al módulo Calculadora de Costos. 2. Visualiza el banner de advertencia de volatilidad. 3. Ingresa filas de insumos (nombre, cantidad, precio unitario). 4. El sistema calcula subtotales y total general en tiempo real (useMemo). 5. Opcionalmente imprime la tabla (window.print()).
Flujos alternativos	El administrador agrega o elimina filas durante el cálculo.
Excepciones	Valores negativos: bloqueados mediante Math.max(0, value). Campos vacíos: el subtotal retorna 0 en lugar de NaN.
Includes	-/-
Extends	-/-
Requerimientos no funcionales	Módulo completamente client-side (React State + useMemo). Sin llamadas a Supabase. Acceso restringido a rol Admin mediante ProtectedRoute. Estilos de impresión (@media print) ocultan navegación y muestran tabla sobre fondo blanco.
Notas	El botón 'Guardar' ejecuta window.print(), no un fetch. Los datos se pierden al refrescar o navegar fuera del módulo.